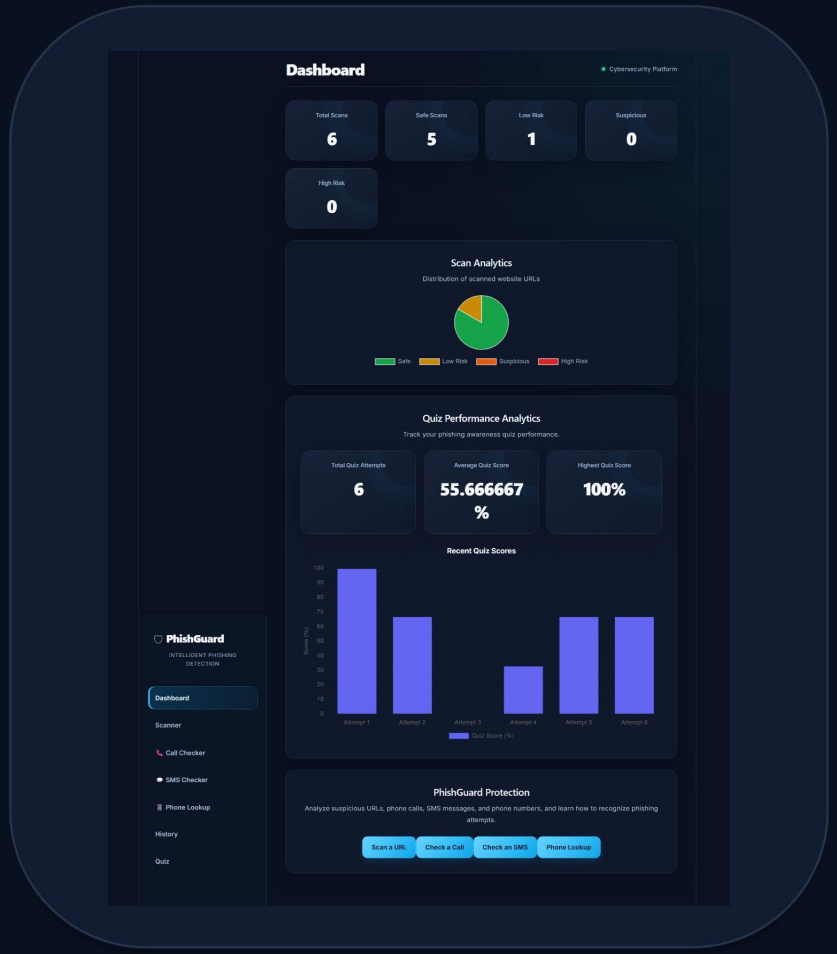
CYBERSECURITY PROJECT

PhishGuard

A Smart Platform for Detecting Phishing & Digital Scams

Frontend • Backend • Database • Security Awareness



Live frontend: phish-guard-rho-green.vercel.app

Project presentation

Project Overview & Key Features

One platform for checking common digital scam signals and improving phishing awareness.

URL SCANNER

Checks website URLs for suspicious patterns such as insecure HTTP, IP-based hosts, shortened links and risky keywords.

CALL CHECKER

Analyses a caller's description for scam language, including OTP requests, PIN requests, payment pressure and threats.

SMS CHECKER

Reviews message text for suspicious links, prizes, OTP requests, account alerts and urgent threats.

PHONE LOOKUP

Checks a phone number against the backend risk assessment and entries in the locally saved directory.

HISTORY

Shows previous URL scan results so users can review scores and risk classifications.

AWARENESS QUIZ

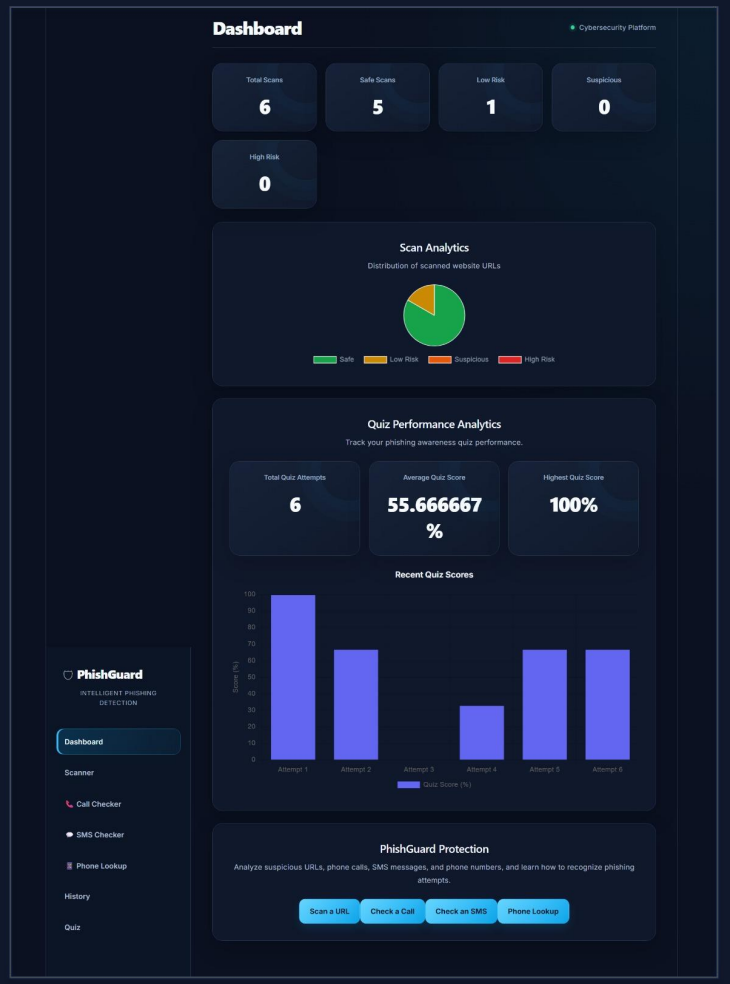
Provides a short phishing-awareness quiz and tracks attempts and scores in dashboard analytics.

Dashboard & Analytics

A central summary of scan activity and phishing-awareness quiz performance.

What the dashboard shows

- Total scans and counts by risk category: Safe, Low Risk, Suspicious and High Risk.
- A pie chart visualises the distribution of scanned website URLs.
- Quiz analytics show total attempts, average score and highest score.
- A recent-scores chart helps users review quiz performance over multiple attempts.
- Quick-action buttons link directly to URL scanning, call checking, SMS checking and phone lookup.

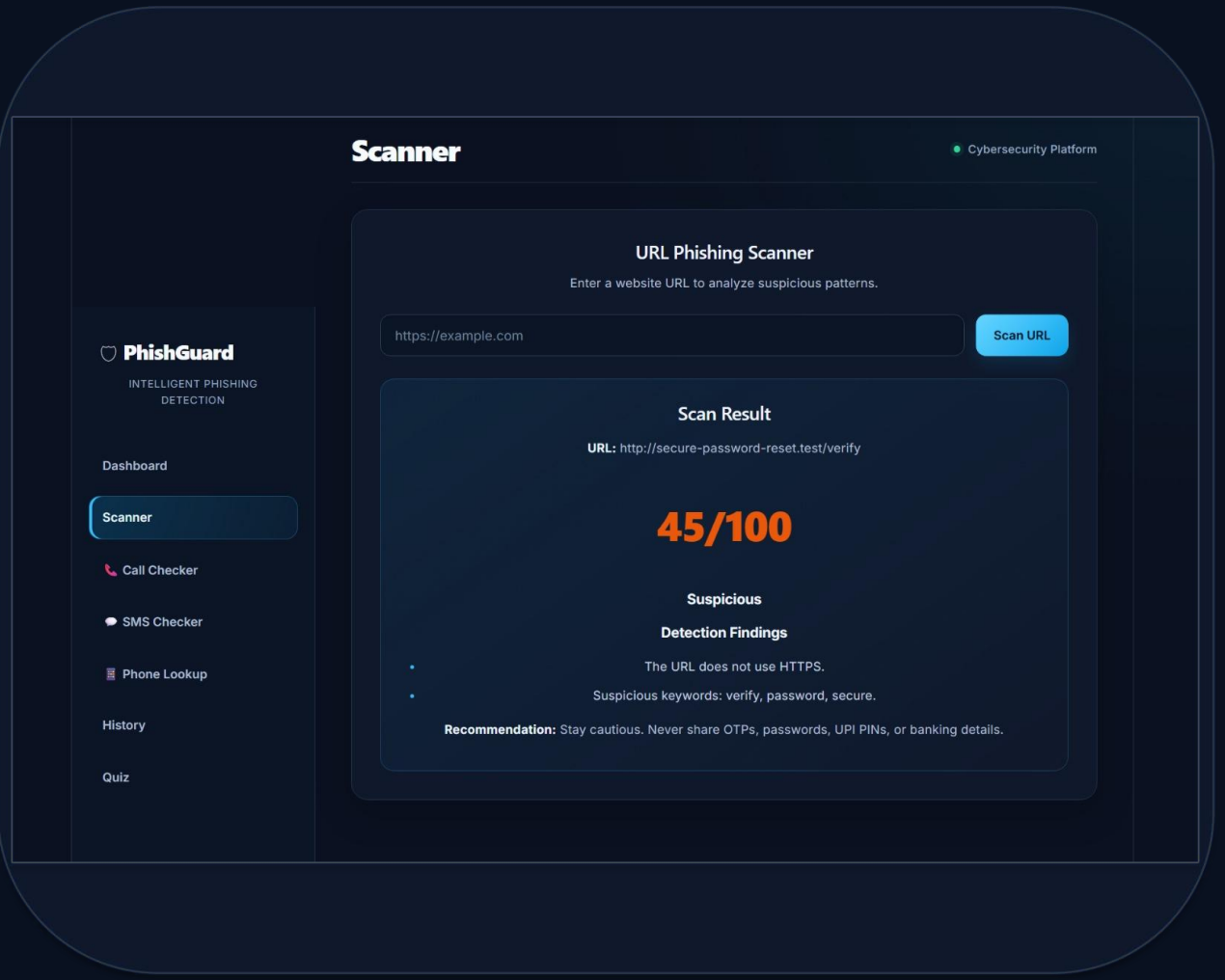


URL Phishing Scanner

Users enter a website address and receive a rule-based risk assessment.

How URL scanning works

- The user submits a URL through the React interface.
- The backend checks URL characteristics such as protocol, IP host, URL shorteners and suspicious keywords.
- Detected indicators contribute points to a score capped at 100.
- The result displays a risk score, status, findings and a safety recommendation.
- Example shown: 45/100 — Suspicious, with HTTP and risky-keyword indicators.

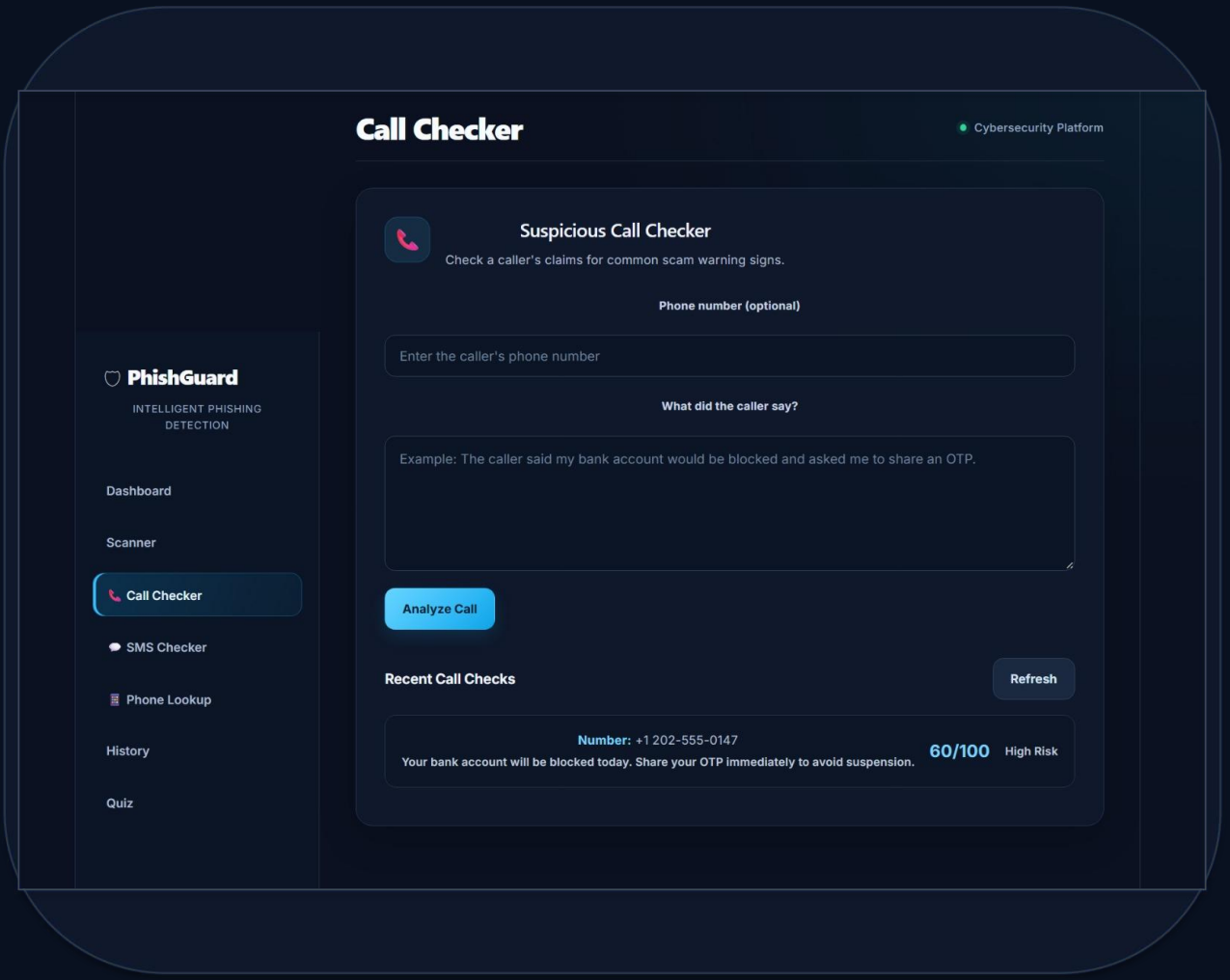


Suspicious Call Checker

A text-based tool to assess warning signs described during a suspicious phone call.

What it analyses

- The phone number is optional; the user describes what the caller said.
- Rules look for OTP or PIN requests, payment pressure, threats and remote-access phrases.
- The backend returns a score, risk status and detected findings.
- Recent call checks are displayed so users can review earlier assessments.
- Example shown: an urgent bank-blocking threat and OTP request scored 60/100 — High Risk.

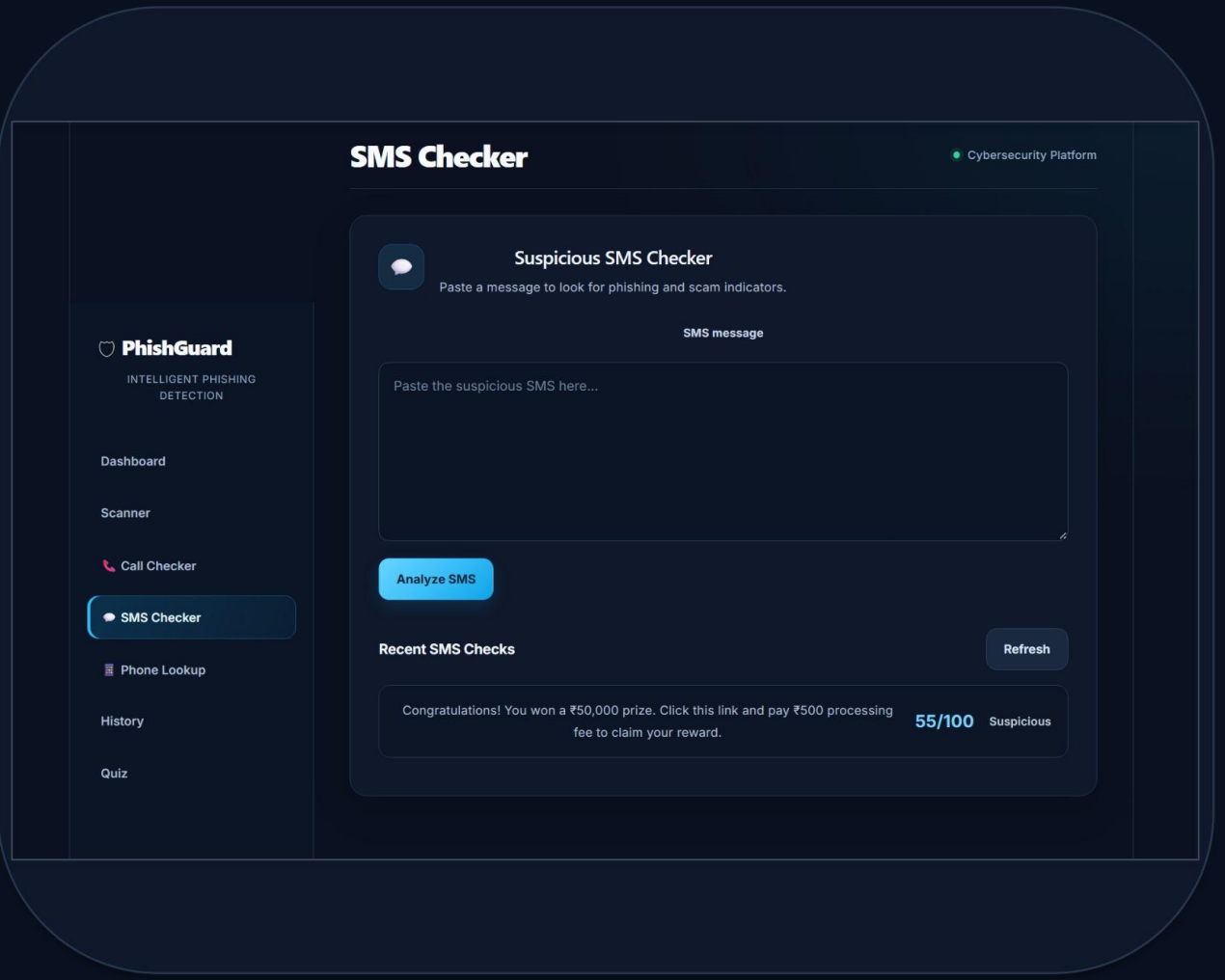


Suspicious SMS Checker

Users paste a message to identify common phishing and scam indicators.

What it analyses

- The user pastes the suspicious SMS into the text area.
- The backend checks for OTP references, suspicious links, prize claims, account alerts and threats.
- Matched indicators add points to the risk score.
- The result and recent SMS checks help users compare message patterns.
- Example shown: a fake prize and processing-fee message scored 55/100 — Suspicious.

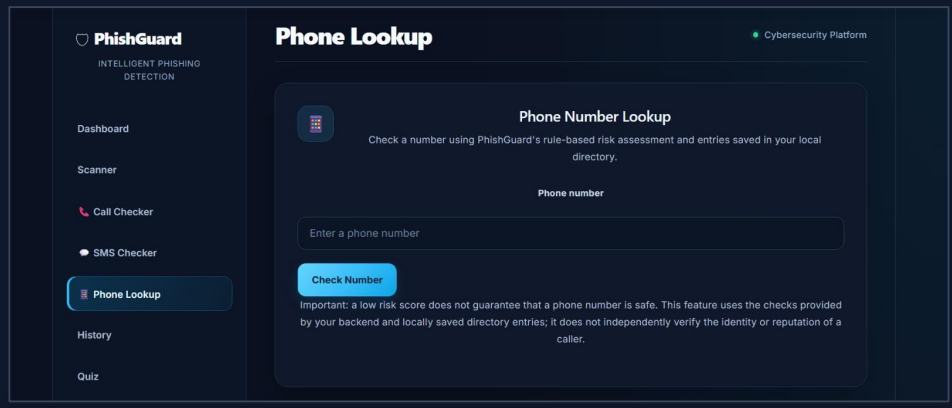


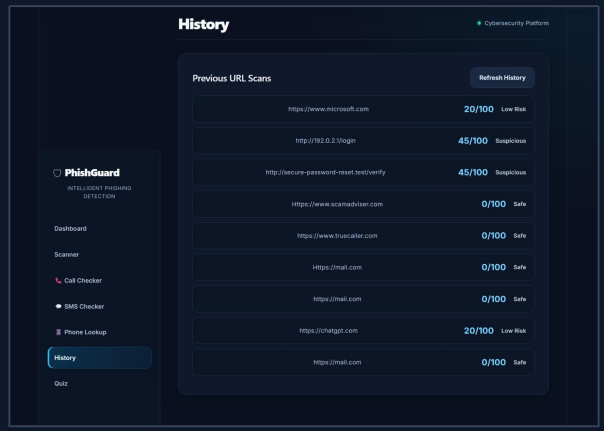
Phone Lookup & Scan History

Supporting tools for checking numbers and reviewing previous URL scan activity.

PHONE NUMBER LOOKUP

- The user enters a phone number and selects Check Number.
- The feature uses the backend's rule-based risk assessment and locally saved directory entries.
- The directory can contain number, business, category and city information.
- A low score does not prove that a number is safe or verify the caller's identity.



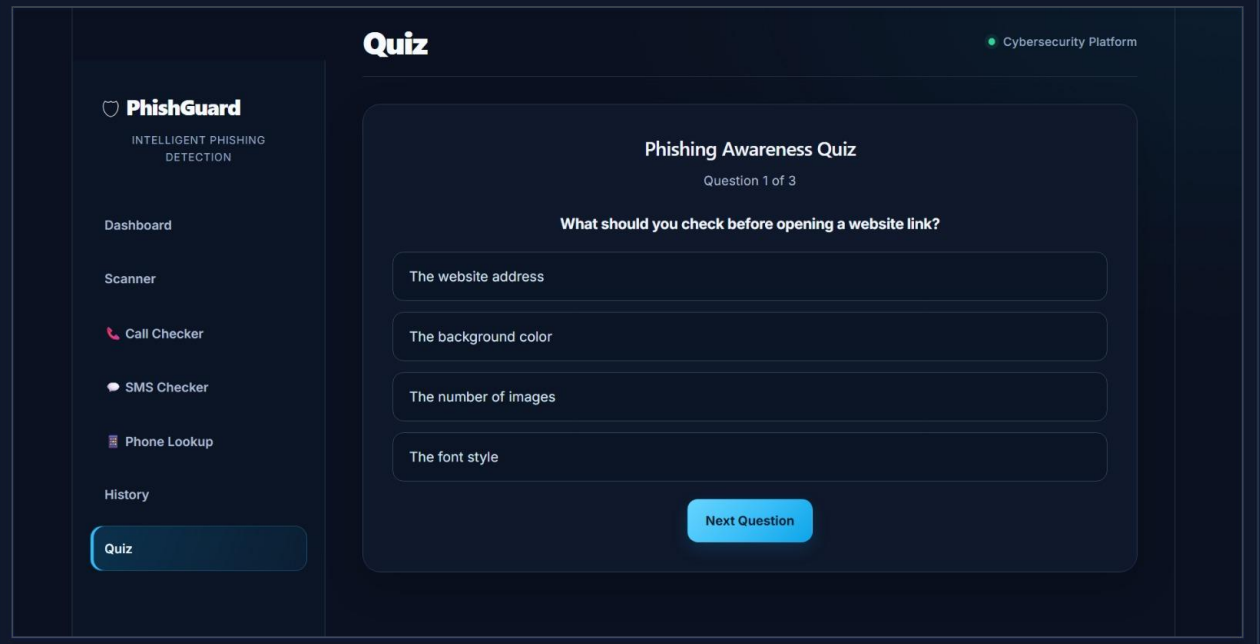


Phishing Awareness Quiz

A learning feature that helps users practise recognising suspicious online activity.

Quiz experience & tracking

- The quiz presents a question with multiple answer choices.
- Users select an answer and continue through the quiz questions.
- Quiz attempts and scores are stored in quiz history.
- The dashboard summarises total attempts, average score and highest score.
- This feature complements automated checks by helping users build safer browsing habits.



System Architecture & Technology Stack

A three-layer design separates the user interface, analysis logic and persistent storage.

01 FRONTEND

React UI

Collects user input, validates forms and displays results and history.

Vercel deployment

02 BACKEND

Node.js + Express

REST APIs, input validation, rule-based scoring and database operations.

mysql2 • CORS • dotenv

03 DATABASE

MySQL / MariaDB

Stores scan records, call and SMS checks, phone lookup data and quiz history.

SQL • connection pool

REQUEST FLOW

User submits input → API validates → Rules calculate risk → Database stores result → JSON response returns to UI

Risk bands: 0–19 Safe | 20–39 Low Risk | 40–59 Suspicious | 60–100 High Risk

Testing, Limitations & Future Scope

Validation, transparent limitations and practical next steps.

TESTING & VERIFICATION

- Postman can be used to send GET and POST requests and inspect JSON responses.
- phpMyAdmin can verify that records are inserted into MySQL tables.
- Example endpoints include /api/test-db, /api/scan, /api/check-call and /api/history.

CURRENT LIMITATIONS

- Detection is rule-based; it is not a trained machine-learning model.
- A low risk score is not a guarantee that a URL, message or phone number is safe.
- Phone lookup relies on backend checks and locally saved directory entries; it does not independently verify identity or reputation.

FUTURE ENHANCEMENTS

- Integrate trusted threat-intelligence feeds.
- Train and evaluate a machine-learning model using suitable datasets.
- Improve phone-number verification and reporting.
- Add authentication, audit controls and monitoring.

PhishGuard provides a first-level warning tool to help users pause and assess suspicious digital interactions.

PHISHGUARD

Thank You!

Thank you for your time and attention.

Questions & Discussion

Live frontend: phish-guard-rho-green.vercel.app